

Practical Part - Q2a

Wednesday, 8 April 2026 17:06

The predicted output of which allele is binding to which peptide will be represented as integers from 0-6 (7 classes). This is because NN can learn numbers with less difficulty than strings of 'A0201' for example. Important note is that before creating the unique labels we sort the classes names in order to always reproduce the same labels no matter who or from which computer it runs.

Regarding the encoding of the peptides which are a specific order of 9 amino acids:

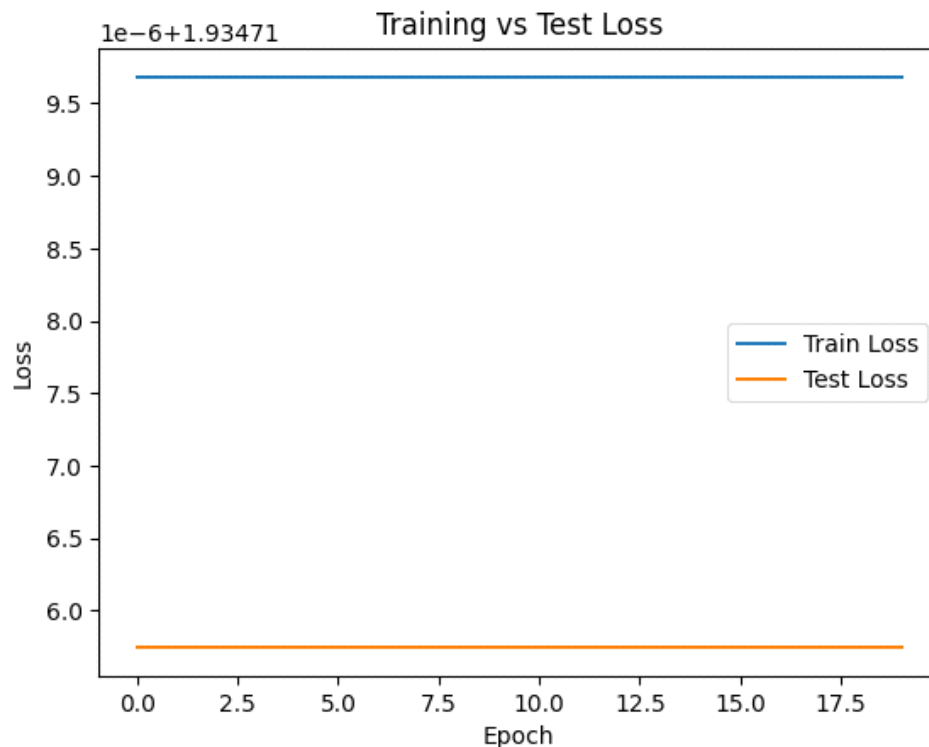
- First of all, we can clearly see that there are 31401 different peptides. This eliminates the idea of having a unique one-hot encoded vector for each peptide - dimensions way too big.
- Second factor to notice is that order matters. If we were to have vectors of size 20 (for each amino acid) and count the amount of times each acid appears in the peptide we can end up with two different peptides having the same vector, which is biologically wrong and can lead to bad learning from the model.
- This leads to the current way we vectorize each peptide. We start with a 9x20 matrix that in fact is a joint of a one-hot encoding for each position in the sequence. The first line of the matrix is a vector of size 20 of 0's other than for the index of the acid that appears in this peptide which will have the value of 1. So for each peptide we have this matrix, we will flatten it and end up with a vector of dimension 180 for each peptide which is unique.

Practical Part - Q2b

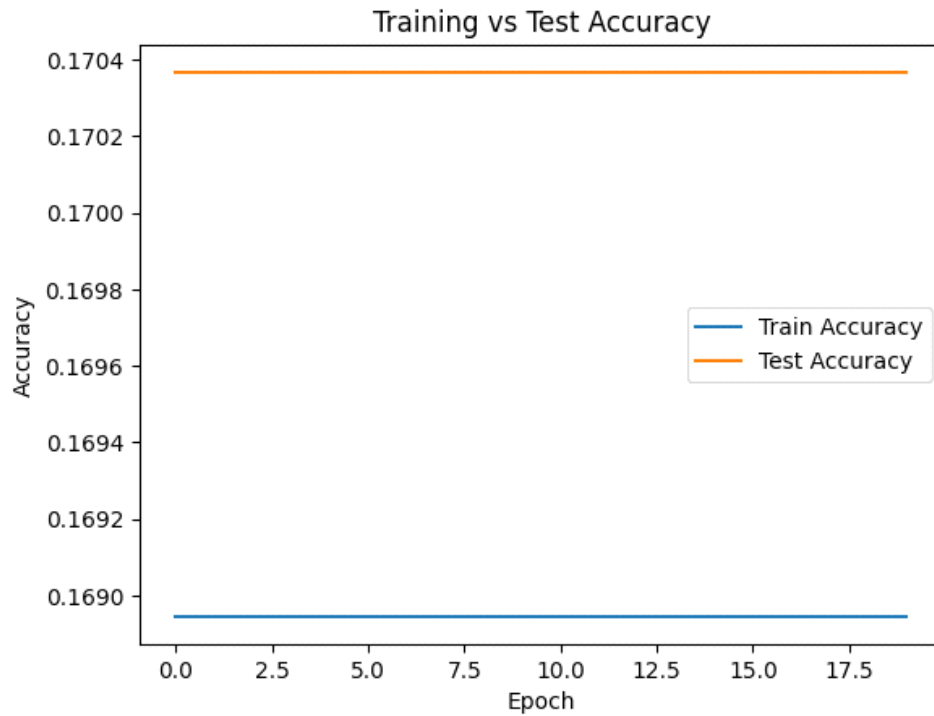
Wednesday, 8 April 2026 17:07

The network input dimension is 180, given that each peptide is represented as a flattened 9x20 one-hot encoded matrix.

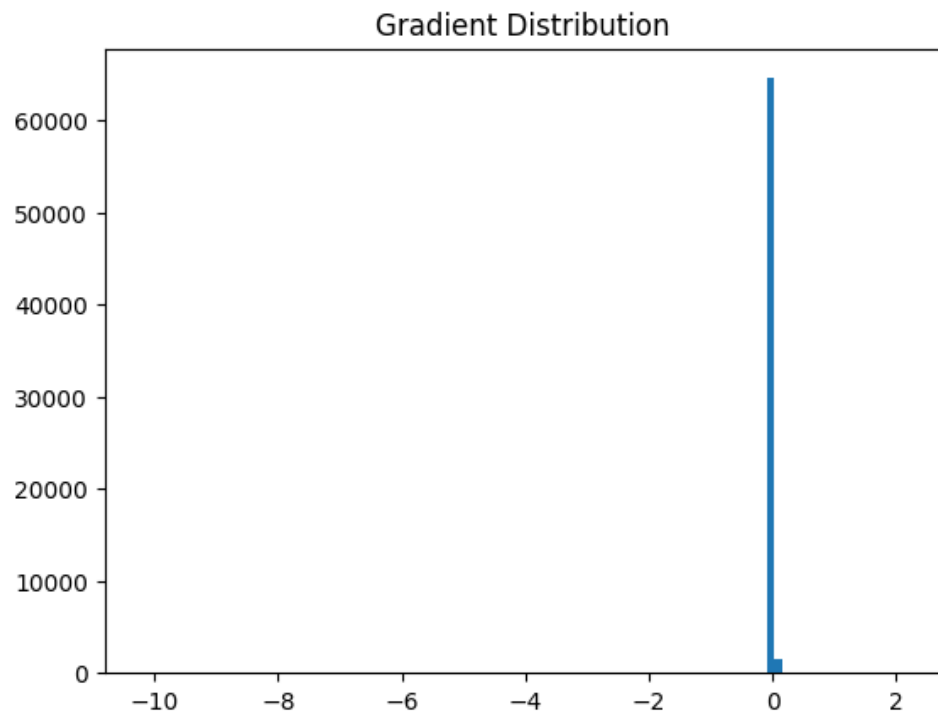
The initial training process showed that both the training and test losses remain constant across epochs, even when increasing the number of epochs. This suggests that the model is not learning.



We also examined the accuracy and observed that it remains very low (~5%) and constant, ruling out the possibility that the model is simply predicting the majority "NEG" class.



To further investigate, we analyzed the gradients during training. The gradient distribution shows that most values are concentrated very close to zero, indicating that the parameter updates are extremely small. As a result, the model weights are barely changing, which explains the constant loss and accuracy.



This behavior can be explained by the input representation. Each input vector has dimension 180, but only 9 entries are non-zero (i.e., only 5% of the features are active). This results in a very sparse representation, meaning that each training example only affects a small subset of the model's parameters. Consequently, the gradients are weak and the learning

process is ineffective.

In addition, the use of cross-entropy loss further amplifies this issue due to the class imbalance in the dataset. Since the "NEG" class dominates the data, the loss is heavily influenced by these samples. As a result, the contribution of rare classes to the overall loss - and therefore to the gradients - is relatively small. Combined with the sparse input representation, this leads to very limited gradient signals, preventing the model from learning meaningful patterns.

Practical Part - Q2c

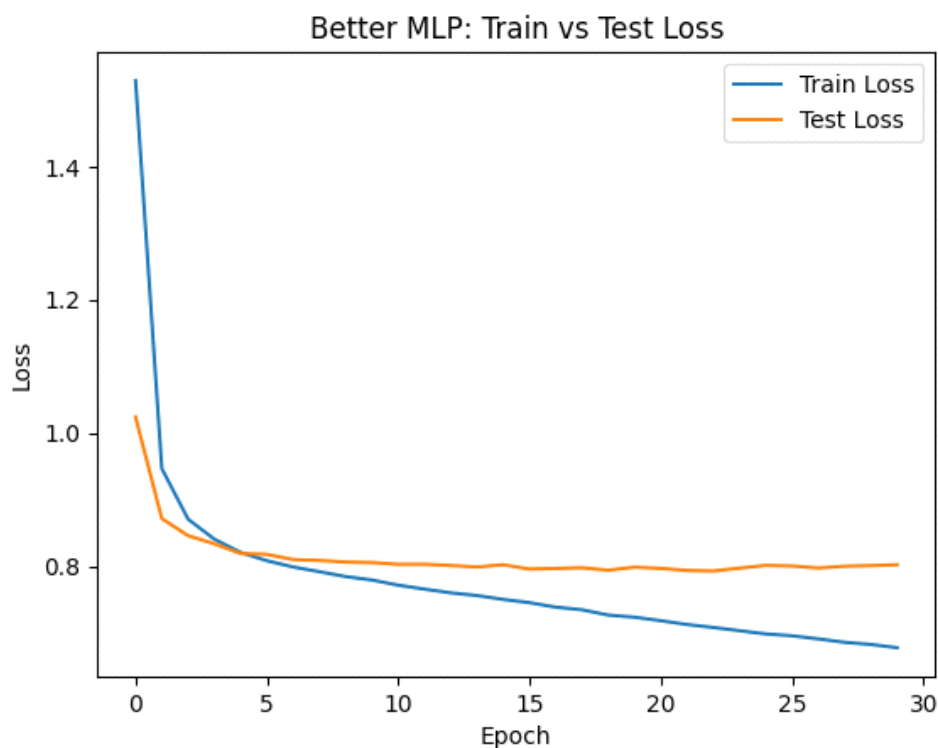
Wednesday, 8 April 2026 17:09

We use an MLP with architecture $180 \rightarrow 32 \rightarrow 16 \rightarrow 7$, where the hidden layers decrease in size. This forces the network to compress the input into a denser internal representation, allowing it to better combine information across positions and extract meaningful patterns from the sparse input.

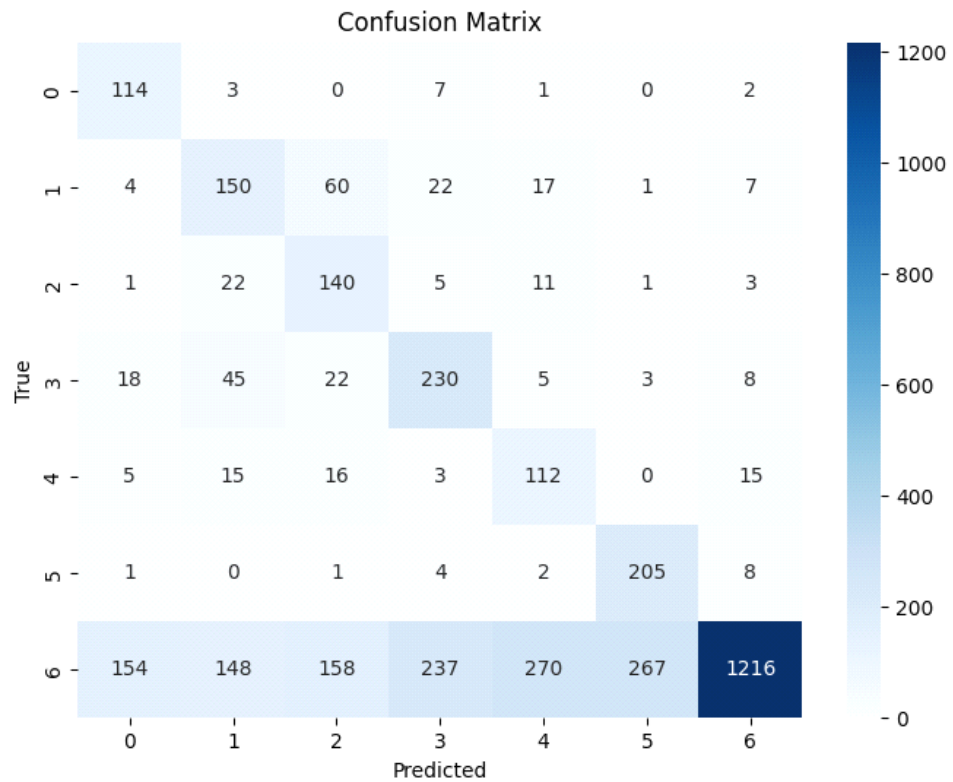
We apply weighted cross-entropy loss to address class imbalance. By assigning higher weights to underrepresented classes, the model increases their influence during training and is encouraged to learn distinctions beyond the dominant "NEG" class.

Training is performed using mini-batches, which provide more diverse and informative gradient updates compared to full-batch training and improve optimization.

From the training curves, the test loss reaches a minimum and then increases, indicating overfitting. We therefore select the model from the best epoch (mid-training) as our final model.



The new architecture successfully learns, which is a major improvement over the initial model. It is able to differentiate between positive classes, but performance on the "NEG" class remains relatively low ($\sim 50\%$). The confusion matrix shows that most misclassifications are biased toward the NEG class, reflecting the strong imbalance in the dataset.



Classes Accuracy:

Class 0 accuracy: 0.898

Class 1 accuracy: 0.575

Class 2 accuracy: 0.765

Class 3 accuracy: 0.695

Class 4 accuracy: 0.675

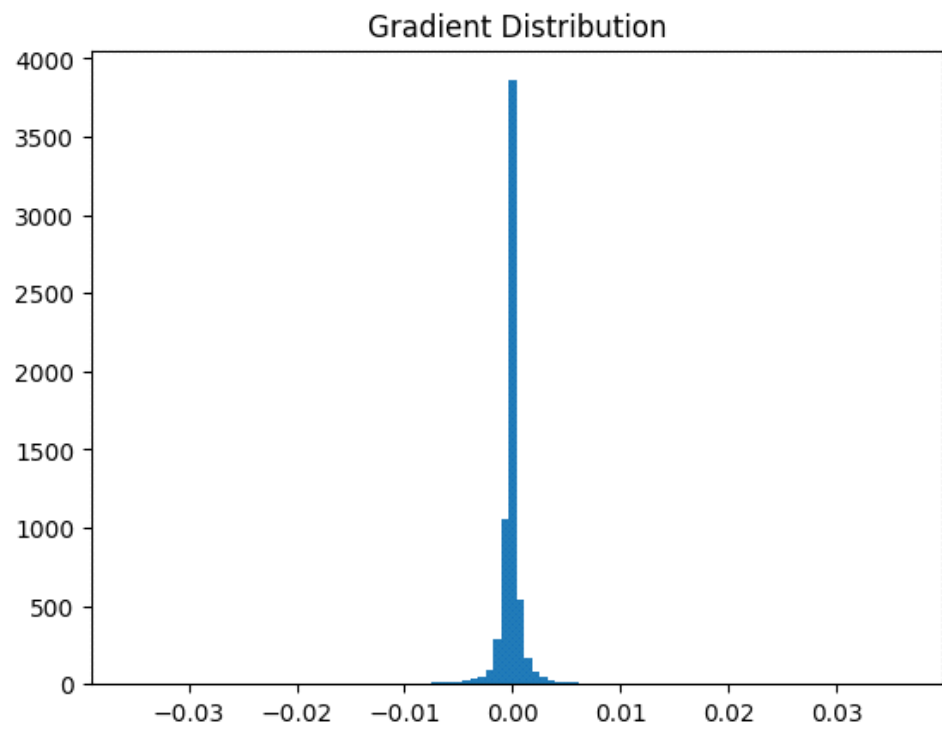
Class 5 accuracy: 0.928

Class 6 accuracy: 0.496

*Where Class 6 is NEG and the rest are the different alleles

Dropout did not improve performance and degraded results.

Overall, the improved architecture enables effective learning, but still exhibits overfitting and sensitivity to class imbalance. Future work could explore better input representations or sequence-aware models.

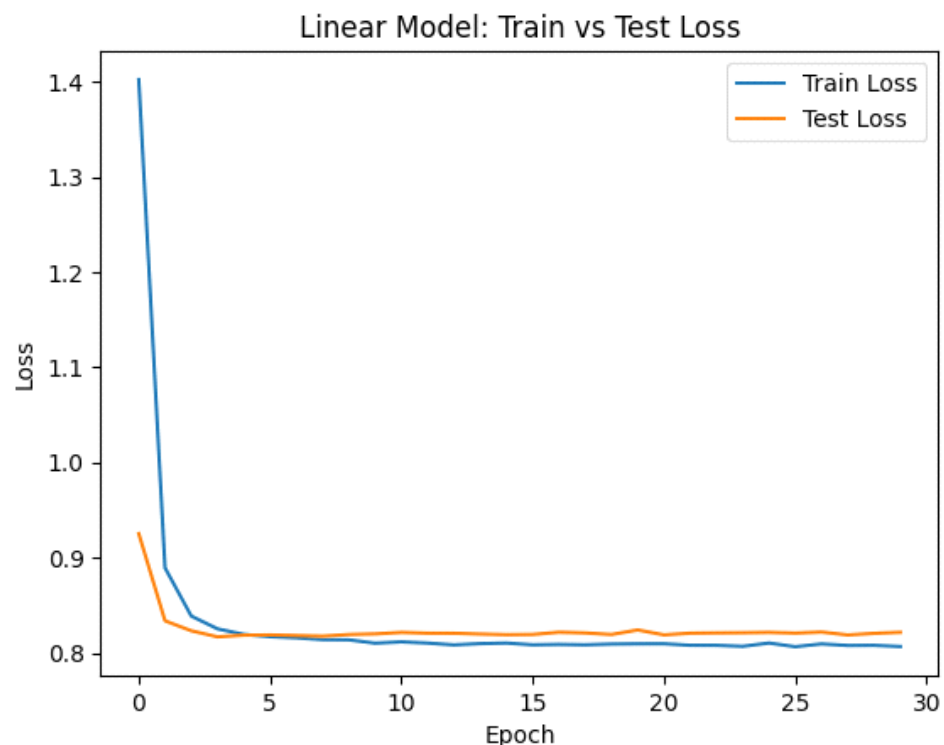


Practical Part - Q2d

Wednesday, 8 April 2026 17:10

We removed all nonlinear activation functions from the network, resulting in a purely linear model. Since a composition of linear layers is equivalent to a single linear transformation, this model effectively reduces to a linear classifier.

The linear model reaches a test loss similar to the nonlinear model (~ 0.8), but its training loss quickly plateaus at a relatively high value and does not continue decreasing. This indicates that the model is underfitting, as it lacks the capacity to further reduce the training error.



In contrast, the nonlinear model continues to reduce the training loss significantly, demonstrating its ability to fit more complex patterns in the data, although at the cost of some overfitting.

This suggests that while the data contains a component that is approximately linearly separable, achieving better performance requires modeling interactions between amino acid positions. A linear model can only capture independent (additive) effects of features, whereas nonlinear models can represent more complex combinatorial patterns, which are likely important for distinguishing between the different peptide classes.

Classes Accuracy:

Class 0 accuracy: 0.882

Class 1 accuracy: 0.636

Class 2 accuracy: 0.710

Class 3 accuracy: 0.686

Class 4 accuracy: 0.711

Class 5 accuracy: 0.932

Class 6 accuracy: 0.482

*Where Class 6 is NEG and the rest are the different alleles

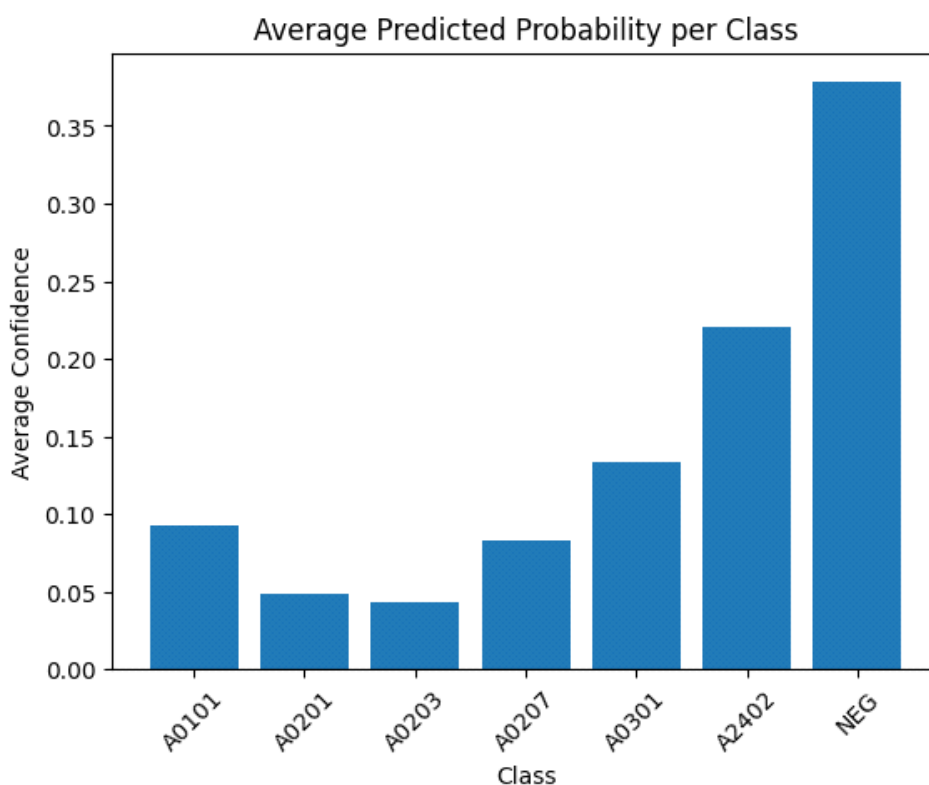
Practical Part - Q2e

Wednesday, 8 April 2026 17:11

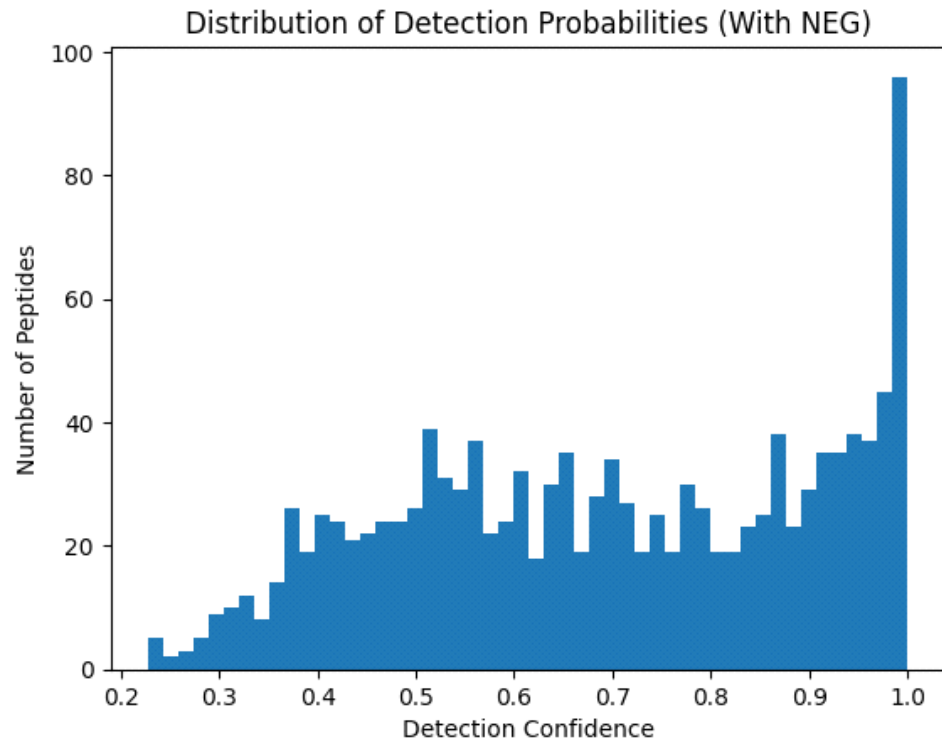
Using the trained MLP, we predicted binding probabilities for all consecutive 9-mer peptides extracted from the SARS-CoV-2 Spike protein. The top three most detectable peptides were:

IKWPWYIWL (A0101, 0.9984)
PYRVVLSF (A0101, 0.9964)
LYRLFRKSN (A0101, 0.9956)

While these predictions exhibit very high confidence, a deeper analysis of the model outputs suggests caution in interpreting the results. The distribution of predicted probabilities shows a strong bias toward the NEG class, which receives both the highest average probability (≈ 0.38) and the largest fraction of predictions ($\sim 40\%$). Among the positive classes, the model tends to assign relatively low and widely spread probabilities, indicating limited confidence and weak separation between alleles.



This behavior is consistent with the class imbalance in the training data, where NEG dominates. As a result, the model appears to struggle with fine-grained discrimination between the different HLA alleles, even if it can somewhat distinguish binding vs. non-binding.



An interesting observation is that all top-3 peptides were assigned to class A0101, despite this class having relatively low average confidence and being one of the least frequently predicted classes ($\sim 2.5\%$). This may suggest that the model has learned a sharper decision boundary for this specific allele, but given the overall instability and bias observed, these predictions should be interpreted with limited trust.

Peptides counts per class:

Class 0: 124

Class 1: 31

Class 2: 42

Class 3: 86

Class 4: 177

Class 5: 300

Class 6: 505

Overall, the analysis indicates that while the model can identify high-confidence candidates, its reliability for allele-specific prediction is constrained by dataset imbalance and model simplicity.

Theoretical Part - Q1

Thursday, 26 March 2026 11:20

Show that the composition of linear functions is a linear function. Show that the composition of affine transformations remains an affine function.

1. Let $f(x) = Ax, g(x) = Bx$,
then $f \circ g(x) = ABx = Cx$ for some matrix C
2. Similarly, for $f(x) = Ax + b, g(x) = Cx + d$,
then: $f \circ g(x) = A(Cx + d) + b = ACx + (Ad + b) = Ex + t$ for some matrix E and some vector t

Theoretical Part - Q2

Thursday, 26 March 2026 11:26

Assume a network is required to predict an angle (0-360 degrees). How will you define a prediction loss that accounts for the circularity of this quantity, i.e., the loss between 2 and 360 is not 358, but 2 (since 0 is 360..). Write your answer in a pseudo-code.

We'll define the following loss function:

$$L(\theta_r, \theta_l) = \min(|\theta_r - \theta_l|, 360 - |\theta_r - \theta_l|)$$

We first note that the abs induces a symmetric loss (since we don't care which angle is smaller/larger, just their difference). Additionally, the min function allows us to always select the acute angle rather than the obtuse.

Theoretical Part - Q3

Thursday, 26 March 2026 11:39

Chain Rule. Differentiate the following terms (specify the points of evaluation of each function):

1. $\frac{\partial}{\partial x} f(x + y, 2x, z)$

Define the following:

$$g(x, y, z) = f(x + y, 2x, z) = f(a_1, a_2, a_3)$$

$$\text{With } a_1 = x + y, a_2 = 2x, a_3 = z$$

Then

$$\frac{\partial g}{\partial x} = \frac{\partial f}{\partial a_1} \cdot \frac{\partial a_1}{\partial x} + \frac{\partial f}{\partial a_2} \cdot \frac{\partial a_2}{\partial x} + \frac{\partial f}{\partial a_3} \cdot \frac{\partial a_3}{\partial x}$$

$$\text{We have: } \frac{\partial a_1}{\partial x} = 1, \frac{\partial a_2}{\partial x} = 2, \frac{\partial a_3}{\partial x} = 0$$

Therefore:

$$\frac{\partial g}{\partial x} = \begin{pmatrix} 1 \\ 2 \\ 0 \end{pmatrix} \nabla f(x + y, 2x, z)$$

Where the evaluation point is $(x + y, 2x, z)$

2. $\frac{\partial}{\partial x} f_1(f_2(\dots f_n(x))) = \frac{\partial}{\partial x} f_1 \circ g(x)$

$$\text{For } g(x) = (f_2 \circ \dots \circ f_n)(x)$$

$$\text{Then } \frac{\partial}{\partial x} f_1 \circ g(x) = Df_1(g(x))Dg(x)$$

Expanding $Dg(x)$:

$$Dg(x) = D(f_2(f_3 \dots (f_n(x)))) \cdot D(f_3 \circ \dots \circ f_n)(x)$$

And Going on recursively, we eventually get:

$$\begin{aligned} \frac{\partial}{\partial x} f_1(f_2(\dots f_n(x))) &= Df_1(f_2 \dots (f_n(x))) \cdot Df_2(f_3 \dots f_n(x)) \dots Df_n(x) \\ &= \prod_{i=1}^n Df_i(f_{i+1} \circ \dots \circ f_n(x)) \end{aligned}$$

3. $\frac{\partial}{\partial x} f_1(x, f_2(x, \dots f_{n-1}(f_n(x))))$

Rewriting this as : $g(x) = f_1(x, u)$, for $u_1 = f_2(x, \dots f_{n-1}(f_n(x)))$

$$\text{Then } \frac{\partial g}{\partial x} = \frac{\partial f_1}{\partial x} + \frac{\partial f_1}{\partial u_1} \cdot \frac{\partial u_1}{\partial x} = \nabla f_1(x) + \frac{\partial f_1}{\partial u} \cdot \frac{\partial u_1}{\partial x}$$

Like in the previous function, we expand recursively $\frac{\partial u}{\partial x}$:

$$\frac{\partial u_1}{\partial x} = \frac{\partial f_2}{\partial x} + \frac{\partial f_2}{\partial u_2} \cdot \frac{\partial u_2}{\partial x} = \dots$$

Until we reach $\frac{\partial u_{n-1}}{\partial x} = \frac{\partial f_n}{\partial x}$

So in total we get:

$$\frac{\partial g}{\partial x} = \frac{\partial f_1}{\partial x} + \frac{\partial f_1}{\partial u_1} \left(\frac{\partial f_2}{\partial x} + \frac{\partial f_2}{\partial u_2} \left(\frac{\partial f_3}{\partial x} + \frac{\partial f_3}{\partial u_3} \left(\dots \frac{\partial f_{n-1}}{\partial x} + \frac{\partial f_{n-1}}{\partial u_{n-1}} \cdot \frac{\partial f_n}{\partial x} \right) \right) \right)$$

4. $\frac{\partial}{\partial x} f(x + g(x + h(x)))$

Defining:

$$u_1 = x + h(x), u_2 = x + g(u_1)$$

$$R(x) = f(u_2)$$

We get:

$$\frac{\partial R}{\partial x} = f'(u_2) \cdot \frac{\partial u_2}{\partial x}$$

$$\text{Expanding } \frac{\partial u_2}{\partial x} = \frac{\partial}{\partial x} (x + g(u_1)) = 1 + g'(u_1) \frac{\partial u_1}{\partial x}$$

Applying the chain rule again:

$$\frac{\partial u_1}{\partial x} = 1 + h'(x)$$

So overall we got:

$$\frac{\partial R}{\partial x} = f'(x + g(x + h(x))) (1 + g'(x + h(x))) (1 + h'(x))$$